



Universidade do Minho
Escola de Engenharia

UNIVERSIDADE DO MINHO

LICENCIATURA EM ENGENHARIA INFORMÁTICA

Trabalho Prático 2

Realizado por:
Diogo Malheiro Pais 107368
José Mário Raimundo Lima 106888
Rodrigo Novais da Silva 108961

1 Introdução

Este trabalho prático teve como objetivo desenvolver e testar um sistema de comunicações entre uma Nave-Mãe, vários rovers e satélites simulando o funcionamento de missões espaciais. Para isso implementámos dois protocolos aplicacionais: o MissionLink, responsável pelo envio e receção de missões sobre UDP, e o TelemetryStream, que transmite telemetria contínua sobre TCP. A Nave-Mãe disponibiliza ainda uma API que permite observar o estado do sistema em tempo real através de um nó adicional chamado Ground Control. Toda esta comunicação foi testada numa topologia criada no CORE, onde configurámos routers que representam satélites e vários rovers distribuídos pela rede. A partir desta simulação foi possível compreender o impacto de múltiplos saltos, atrasos e possíveis perdas de pacotes no desempenho dos protocolos e na estabilidade geral do sistema.

2 Topologia da Rede

A topologia utilizada neste trabalho está representada na Figura 1 e foi construída no ambiente CORE de forma a simular uma rede com várias zonas distintas e vários saltos entre rovers e a Nave-Mãe. A Nave-Mãe encontra-se numa zona central da topologia e desempenha o papel de ponto de coordenação e receção de dados. À sua volta distribuímos um conjunto de routers que representam satélites ou nós de retransmissão, criando vários caminhos alternativos e garantindo que nenhuma comunicação ocorre de forma direta e simples. Cada rover está colocado numa extremidade da rede de forma a maximizar o número de saltos necessários para chegar à Nave-Mãe, o que torna o cenário especialmente interessante para testar comunicação UDP e TCP em simultâneo.

Através desta topologia foi possível observar como a comunicação varia consoante o caminho percorrido pelos pacotes. Como existem vários routers e ligações entre eles, cada fluxo pode atravessar um número diferente de saltos e, por isso, sofrer variações de latência e eventuais perdas. Isto obrigou o MissionLink a controlar melhor as confirmações e retransmissões, especialmente porque pequenos atrasos podiam ser suficientes para provocar timeouts ou fazer com que pacotes chegassem fora de ordem. Também foi possível perceber como o TelemetryStream reagia quando vários rovers estavam a enviar telemetria ao mesmo tempo, verificando a forma como o TCP lidava naturalmente com congestionamento e com a gestão de várias ligações simultâneas.

A inclusão de IPv4 e IPv6 permitiu ainda explorar diferentes tipos de endereçamento e garantiu que a topologia cumpria todos os requisitos definidos no enunciado. O Ground Control, ligado diretamente à Nave-Mãe, ajudou a validar rapidamente o estado do sistema, uma vez que permitia observar em tempo real não só a posição e o estado de cada rover, mas também o progresso das missões e o funcionamento dos protocolos que implementamos. No conjunto, esta topologia serviu como um campo de testes bastante completo, onde foi possível analisar tanto o comportamento dos nossos protocolos como o impacto das configurações de rede aplicadas no CORE.

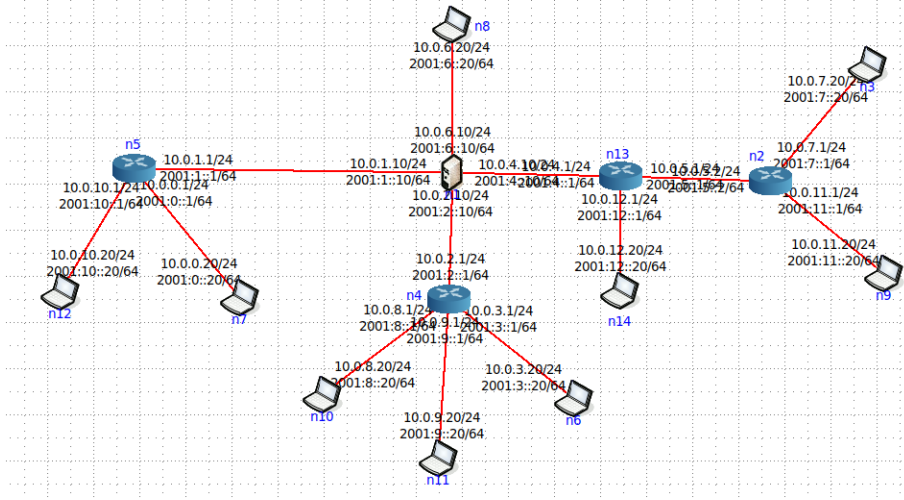


Figura 1: Topologia implementada no CORE com roteadores satélite, rovers distribuídos pela rede e Nave-Mãe no centro.

3 Protocolo MissionLink (ML)

O MissionLink foi desenvolvido como o protocolo responsável pela comunicação crítica entre a Nave-Mãe e os rovers, utilizando UDP. Como o UDP não garante a entrega nem a ordem dos pacotes, foi necessário implementar mecanismos de fiabilidade a nível aplicacional. Cada mensagem enviada contém um número de sequência que permite ao recetor confirmar exatamente qual foi a mensagem recebida, e sempre que o emissor não obtém uma resposta dentro do tempo previsto, a mensagem é retransmitida. Na prática, o envio é repetido até cinco vezes, e só é considerado bem-sucedido quando chega um ACK com o mesmo número de sequência.

O servidor processa cada pacote recebido, valida o seu conteúdo e envia um ACK correspondente para evitar que o rover continue a reenviar a mesma mensagem. Esta lógica está presente no método `listen_packet`, que

trata tanto mensagens normais como confirmações. Durante os testes realizados no CORE foi possível observar que estes mecanismos mantiveram a comunicação estável mesmo quando existiam atrasos e perdas na rede. O MissionLink acabou por garantir que as missões eram entregues e que as atualizações chegavam de forma consistente, compensando as limitações naturais do UDP.

3.1 Serialização e Formato das Mensagens

Para cumprir os requisitos de eficiência da missão e evitar o envio de texto em claro na rede, foi implementado um mecanismo de serialização binária personalizado (módulo `serializer.py`). Todas as mensagens trocadas via MissionLink são convertidas para arrays de bytes antes da transmissão, garantindo compactação e rapidez de processamento.

O cabeçalho das mensagens possui uma estrutura fixa que permite ao recetor saber como processar o *payload*. A estrutura geral é a seguinte:

Tabela 1: Estrutura do Cabeçalho MissionLink (Binário)

Campo	Tamanho	Descrição
Tipo	1 Byte	Identifica a mensagem: 0 (Missão), 1 (Update), 2 (Final), 3 (Pedido), 4 (ACK).
Sequência	1 Byte	Número sequencial para controlo de fluxo e duplicados.
Len ID	1 Byte	Comprimento da string do ID do Rover.
Len TS	1 Byte	Comprimento da string de Timestamp.
Len Payload	2 Bytes	Tamanho total em bytes do conteúdo da mensagem.
Rover ID	Variável	Identificador do remetente/destinatário.
Timestamp	Variável	Registo temporal da emissão.
Payload	Variável	Dados específicos da missão (ver secção seguinte).

3.2 Estrutura do Payload

O conteúdo do campo **Payload** varia consoante o tipo de mensagem, transportando os dados essenciais da missão de forma serializada:

- **Mensagem de Missão (Tipo 0):** Contém a definição completa da tarefa a realizar. Os bytes estão organizados sequencialmente com:
 - **ID da Missão:** String que identifica a missão (ex: "M-AM-01").
 - **Tipo de Tarefa:** Identificador da operação (ex: "AnaliseAmbiental").

- **Duração:** Inteiro que define o tempo limite em segundos.
- **Coordenadas:** Pares de valores (x, y) que delimitam a área de atuação.
- **Mensagem de Update (Tipo 1):** Transporta os dados recolhidos pelo Rover. Inclui o estado atual da tarefa e valores de sensores (ex: temperatura, número de fotos tiradas) serializados conforme a missão específica.

Esta decisão de design permitiu um controlo total sobre o tráfego de rede, evitando o *overhead* associado a formatos como JSON ou XML nas ligações críticas da missão.

4 Protocolo TelemetryStream (TS)

O protocolo TelemetryStream foi desenhado para a transmissão contínua de dados de monitorização, tirando partido das garantias de entrega e ordem oferecidas pelo protocolo TCP.

4.1 Arquitetura Cliente-Servidor

Na nossa implementação, a Nave-Mãe atua como servidor TCP capaz de lidar com múltiplas conexões simultâneas (implementado em `telemetry_server.py`). Utilizamos *multithreading* para que o servidor possa aceitar novos rovers sem bloquear a receção de dados dos rovers já conectados. Cada rover, ao iniciar, estabelece uma ligação persistente com a Nave.

4.2 Mecanismos de Robustez

Para garantir que a monitorização não falha em cenários de rede instável, o cliente de telemetria (no Rover) implementa uma lógica de **reconexão automática**. O código opera num ciclo infinito onde, caso a ligação TCP caia (detetado por exceção no envio/receção), o Rover aguarda 5 segundos e tenta restabelecer a conexão com a Nave-Mãe. Isto assegura que, mesmo após uma falha temporária num satélite, o fluxo de dados é retomado automaticamente.

5 Especificação da API de Observação

A Nave-Mãe expõe uma API HTTP para permitir a visualização externa do estado da missão pelo nó Ground Control. Esta API foi desenhada para ser simples, stateless e baseada em JSON.

5.1 Endpoints Definidos

A API disponibiliza um único endpoint de consulta, acessível via método HTTP GET:

- **URL:** `http://<IP-NAVE>:5000/api/status`
- **Descrição:** Retorna um snapshot completo do estado atual do sistema, incluindo a última telemetria recebida de todos os rovers conectados e o histórico de todas as missões (ativas e concluídas).

5.2 Estrutura das Mensagens (Exemplo)

Abaixo apresenta-se um exemplo real da resposta JSON gerada pela API, contendo a secção de telemetria (TCP) e a secção de missões (UDP):

Listing 1: Exemplo de Resposta JSON da API

```
1 {
2   "telemetry": {
3     "R-12": {
4       "bateria": 85.0,
5       "temperatura": 31.2,
6       "coordenadas": [10, 20],
7       "estado": "AnaliseAmbiental",
8       "velocidade": 0.5
9     }
10  },
11  "missions": [
12    {
13      "mission_id": "M-AM-01",
14      "rover_id": "R-12",
15      "type": "AnaliseAmbiental",
16      "status": "Active",
17      "start_time": "09/12/2025-16:56:40",
18      "updates": [
19        {
20          "kind": "UpdateAmbiental",
21          "temperatura": 24,
22          "pos": [10, 20]
23        }
24      ]
25    }
26  ]
27 }
```

5.3 Implementação e Decisões de Design do Ground Control

Para o componente Ground Control, optámos por desenvolver uma aplicação de linha de comandos (CLI) em Python, executada num nó independente da topologia (n8).

Esta decisão de design baseou-se em três fatores principais:

1. **Leveza e Compatibilidade:** Uma interface CLI consome poucos recursos e integra-se nativamente com os terminais Linux dos nós virtuais do CORE, evitando a complexidade de configurar servidores web gráficos dentro da emulação.
2. **Polling e Atualização:** A aplicação funciona num ciclo contínuo, onde o utilizador seleciona opções num menu interativo. A cada pedido, o Ground Control realiza um pedido HTTP GET à API da Nave-Mãe, garantindo que a informação apresentada é sempre a mais recente.
3. **Visualização Estruturada:** O JSON recebido é processado e formatado em tabelas ASCII legíveis, permitindo ao operador visualizar rapidamente o estado crítico dos rovers (bateria, temperatura) e o progresso das missões, conforme ilustrado na Figura [2](#).

```
Opção > 1
=== ESTADO DOS ROVERS ===

ID          | BAT   | TEMP  | POS          | ESTADO
-----
R-12        | 90%   | 33C   | [5, 14]      | M-RND-001
R-9         | 94%   | 32C   | [5, 5]       | M-FT-02
R-11        | 94%   | 32C   | [15, 15]     | M-CL-03

[Enter] Voltar ao menu...

GROUND CONTROL
1 - Ver Estado dos Rovers
2 - Ver Lista de Missões
3 - Detalhes de uma Missão
0 - Sair

Opção > 2
=== HISTÓRICO E MISSÕES ATIVAS ===

ID          | Rover  | Tipo          | Status
-----
M-RND-003   | R-11   | CapturaImagens | Active
M-RND-002   | R-9    | ColetaAmostras | Active
M-RND-001   | R-12   | CapturaImagens | Active
M-CL-03     | R-11   | ColetaAmostras | Completed
M-FT-02     | R-9    | CapturaImagens | Completed
M-AM-01     | R-12   | AnaliseAmbiental | Completed

[Enter] Voltar ao menu...

GROUND CONTROL
1 - Ver Estado dos Rovers
2 - Ver Lista de Missões
3 - Detalhes de uma Missão
0 - Sair

Opção > 3

Digite o ID da Missão: M-AM-01

--- Detalhes de M-AM-01 ---
Rover Responsável: R-12
Tipo de Missão:    AnaliseAmbiental
Estado Atual:      Completed
Updates Recebidos: 6

[Enter] Voltar ao menu...[]
```

Figura 2: Interface de Linha de Comandos (CLI) do Ground Control a apresentar o estado dos rovers e o histórico de missões.

6 Guia de Execução

Para executar a simulação e os componentes desenvolvidos, é necessário garantir que a estrutura de pastas está correta, nomeadamente a pasta principal deve chamar-se **este** para que as importações de módulos Python funcionem.

6.1 Arranque do Ambiente CORE

1. Abrir o emulador CORE com privilégios de superutilizador:

```
sudo core-gui
```

2. Carregar o ficheiro de topologia `CC_TP2.imn` ou `CC_TP2_loss.imn` localizado na pasta do projeto.
3. Pressionar o botão **Start** (ícone verde) para iniciar a emulação da rede.

6.2 Execução dos Componentes

Após a rede estar ativa, deve-se abrir terminais (*Shell Window*) nos nós respetivos e executar os comandos pela ordem indicada:

1. **Nave-Mãe (Nó Central):**

```
cd ~/.../TP2/este/Nave
python3 Nave.py
```

2. **Ground Control (Nó Vizinho):**

```
cd ~/.../TP2/este
python3 ground_control.py
```

3. **Rovers (Nós nas extremidades):** Para iniciar um rover, deve especificar-se o seu ID numérico como argumento (ex: ID 12).

```
cd ~/.../TP2/este/Rover
python3 rover.py 12
```

7 Resultados dos Testes

O sistema foi validado executando a topologia no CORE e submetendo os protocolos a diferentes condições de stress.

7.1 Cenários de Teste

- **Robustez e Recuperação de Falhas (MissionLink):** Configurámos os links dos satélites com injeção de perda de pacotes e latência, utilizando a topologia de stress. Observou-se que o protocolo lidou eficazmente com perdas pontuais através de retransmissões. Mais importante ainda, em situações onde o limite de tentativas foi atingido, o sistema demonstrou robustez: os rovers detetaram a falha crítica, abortaram a operação localmente e reiniciaram o ciclo de pedido de missão automaticamente, evitando estados de bloqueio (*deadlocks*). Este comportamento de autorrecuperação pode ser observado nos logs da Figura 3.
- **Concorrência (Múltiplos Rovers):** Testámos o sistema com 3 rovers simultâneos. A Nave-Mãe, utilizando threads separadas e mecanismos de *Lock* nas estruturas de dados partilhadas, conseguiu processar os pedidos de missão via UDP e os fluxos de telemetria TCP sem corrupção de dados ou bloqueios.
- **Reconexão (TelemetryStream):** Ao desligar interfaces de rede durante a execução, confirmámos que os rovers entravam em modo de reconexão. Assim que a rota era restabelecida, o Ground Control voltava a receber dados atualizados automaticamente.

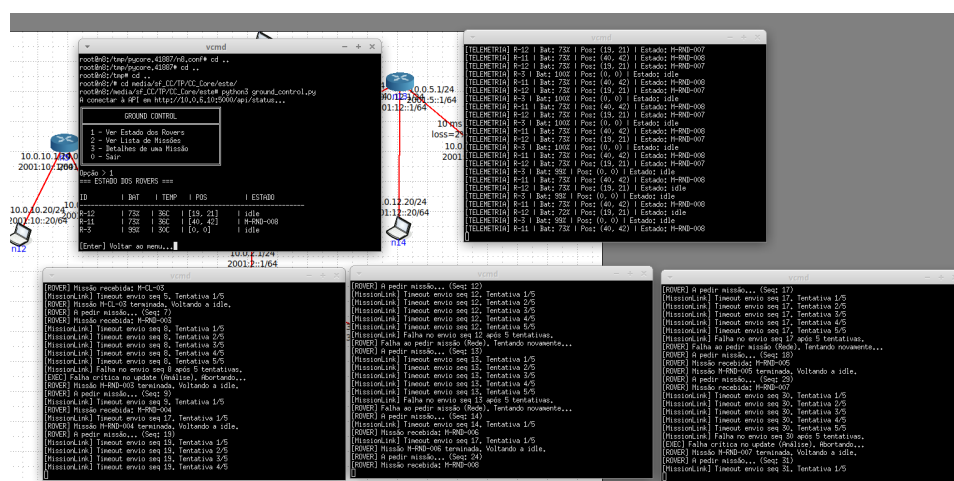


Figura 3: Prova de Recuperação de Falhas: Os terminais mostram os Rovers a detetar falhas de envio (após 5 tentativas), a abortar a missão corrente e a reiniciar automaticamente o pedido de nova missão (linhas "[ROVER] A pedir missão..."), garantindo a continuidade do serviço mesmo em cenários de degradação da rede.

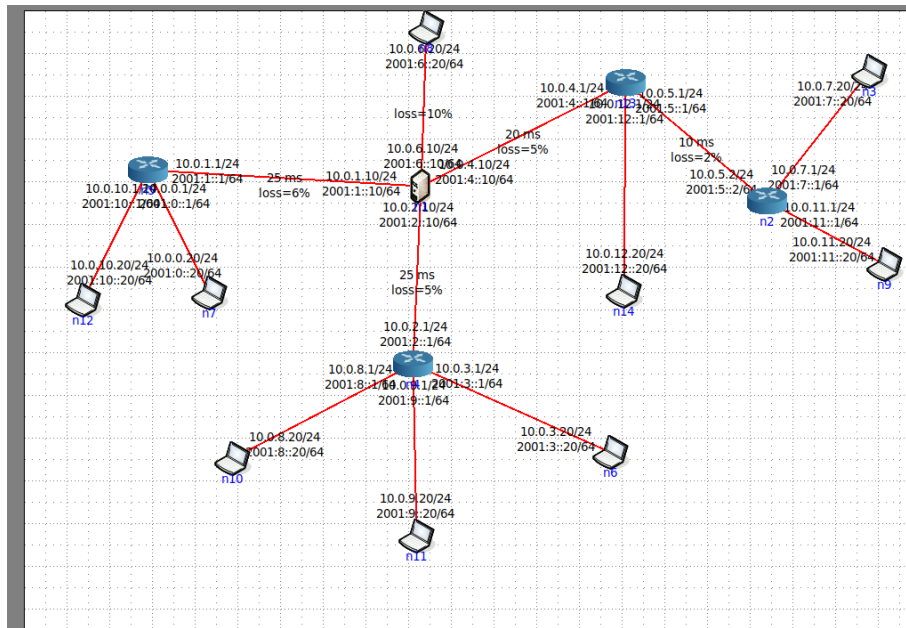


Figura 4: Configuração de links com injeção de erros e latência para testes de robustez.

8 Reflexão Crítica e Conclusão

O desenvolvimento deste trabalho permitiu implementar com sucesso uma arquitetura de comunicação espacial robusta. No entanto, identificámos algumas limitações e oportunidades de melhoria:

- **Limitação do Stop-and-Wait:** Embora robusto, o protocolo MissionLink espera pela confirmação de cada pacote antes de enviar o próximo. Em cenários reais com latência espacial muito elevada (segundos ou minutos), isto tornaria a comunicação ineficiente. Uma melhoria seria implementar Janelas Deslizantes (*Sliding Window*) para permitir múltiplos pacotes em trânsito.
- **Endereçamento Estático:** A implementação atual depende de endereços IP fixos configurados no código dos Rovers. Uma abordagem mais flexível envolveria um mecanismo de descoberta de serviços (Service Discovery) para que os Rovers pudessem localizar a Nave-Mãe dinamicamente na rede.
- **Segurança:** A serialização binária atual não inclui encriptação. Qualquer nó intermediário (satélite) pode ler o conteúdo das mensagens. A implementação de TLS para o TCP e encriptação do payload UDP seria fundamental num cenário real.

Em conclusão, a combinação de um protocolo fiável sobre UDP para comandos críticos e TCP para monitorização contínua demonstrou ser uma solução eficaz, cumprindo todos os requisitos propostos e garantindo a observabilidade total da missão através da API e do Ground Control.